VECTORS AND HASHMAPS

Scalar Types

Integers, float, bool, char

Compound Types

Tuples, Arrays

Function Types

Functions

Collection Types

Vec, String, Hashmap, …

Custom Types

Enum, struct

Special Types

Unit types, &T, &mut T

Smart pointers

Box, Rc, Arc, RefCell, …

Collection Types

Type	Description
`Vec<T>`	Growable array (vector) holding elements of type `T`
`String`	Growable UTF-8 encoded text string
`HashMap<K, V>`	Unordered key-value store using hashing

I. VECTORS

Definition

A **vector** is a **growable, ordered collection** of values in Rust.
It stores elements of the **same type** and allows you to
dynamically add, remove, or access items at runtime.

What we have seen so far (arrays)

```rust
let fruits: [&str; 3] = ["Apple", "Banana", "Cherry"];
```

With vectors

```
let fruits = vec!["Apple", "Banana", "Cherry"];
```

```rust
let fruits: [&str; 3] = ["Apple", "Banana", "Cherry"];
```

Arrays

```rust
let fruits = vec!["Apple", "Banana", "Cherry"];
```

Vectors

Push, pop, etc.


ART DECO HOUSE
THE FASHION BOOK
Swiss Graphic Design
pierre cardin
NEW WAYS WITH PAINT
AFRO MODERN

Use cases of vectors

Using a `Vec` provides a **flexible, growable collection** that can change size at **runtime**. Unlike arrays, which have a fixed size, vectors allow you to dynamically **add, remove, or modify** elements as your program runs.

This makes `Vec` ideal for working with data when the number of items **isn't known in advance or can vary** — such as user input, file lines, or network data. Vectors are also widely supported by Rust's standard library.

Exercice 1:
Vector basics

Common Methods of Vectors in Rust

Vectors in Rust provide several useful methods for working with their contents. Here are some common methods:

- `len()`: Returns the number of elements in the vector.
- `is_empty()`: Returns true if the vector is empty, false otherwise.
- `contains()`: Returns true if the vector contains a specific element, false otherwise.
- `iter()`: Returns an iterator over the vector's elements.
- `into_iter()`: Consumes the vector and returns an iterator over its elements.
- `push()`: Adds an element to the end of the vector.
- `pop()`: Removes and returns the last element of the vector.
- `remove()`: Removes the element at the specified index and returns it.
- `get()`: Returns a reference to the element at the specified index, or None if the index is out of bounds.
- `get_mut()`: Returns a mutable reference to the element at the specified index, or None if the index is out of bounds.
- `sort()`: Sorts the elements of the vector in ascending order.
- `reverse()`: Reverses the order of the elements in the vector.
- `clear()`: Clears the vector, removing all values.

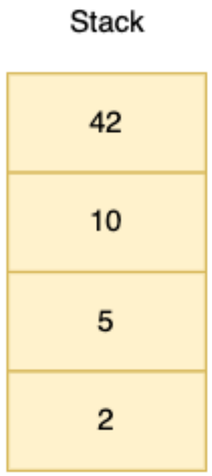
Exercice 2:
Vector access elements

Arrays

Arrays are one of the first data types beginner programmers learn. An array is a collection of elements of the same type allocated in a contiguous memory block. For example, if you allocate an array like this:

```
let array: [i32; 4] = [42, 10, 5, 2];
```

Then all the `i32` integers are allocated next to each other on the stack:



Vectors

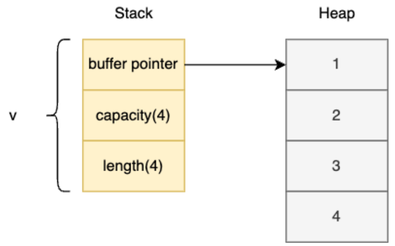
The big limitation of arrays is that they are fixed in size. In contrast, vectors can grow at runtime:

```
fn main() {
    //There are three elements in the vector initially
    let mut v: Vec<i32> = vec![1, 2, 3];
    //prints 3
    println!("v has {} elements", v.len());
    //but you can add more at runtime
    v.push(4);
    v.push(5);
    //prints 5
    println!("v has {} elements", v.len());
}
```

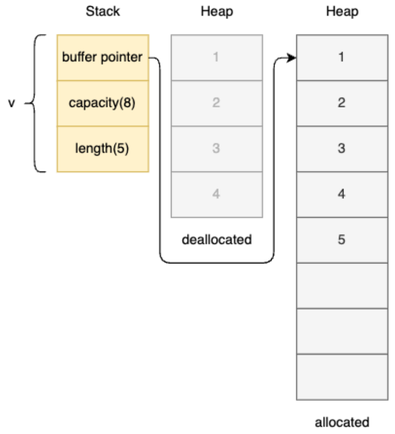
How does a vector allow dynamic growth? Internally, a vector keeps all the elements in an array allocated on the heap. When a new element is pushed, the vector checks if there is still some capacity left in the array. If not, the vector allocates a bigger array, copies all the elements to the new array and deallocates the old array. This can be seen in the following code:

```
fn main() {
    let mut v: Vec<i32> = vec![1, 2, 3, 4];
    //prints 4
    println!("v's capacity is {}", v.capacity());
    println!("Address of v's first element: {:p}", &v[0]);//{:p} prints th
    v.push(5);
    //prints 8
    println!("v's capacity is {}", v.capacity());
    println!("Address of v's first element: {:p}", &v[0]);
}
```

Initially the capacity of the `v`'s backing array is 4:



A new element is then pushed onto the vector. This makes the vector copy all the elements to a new backing array of capacity 8:



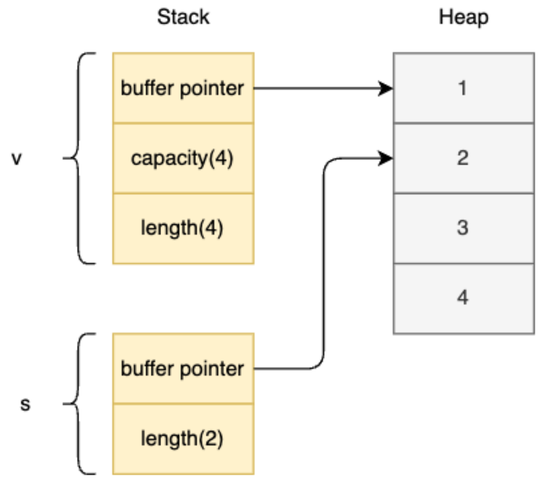
The program also prints the address of the first element of the array before and after pushing a new element onto the vector. Both these printed addresses will be different from each other. This change in addresses is clear evidence of a new array of size 8 being allocated behind the scenes.

Slices of vectors

For example, if you create a slice to a vector:

```
let v: Vec<i32> = vec![1, 2, 3, 4];
let s = &v[1..3];
```

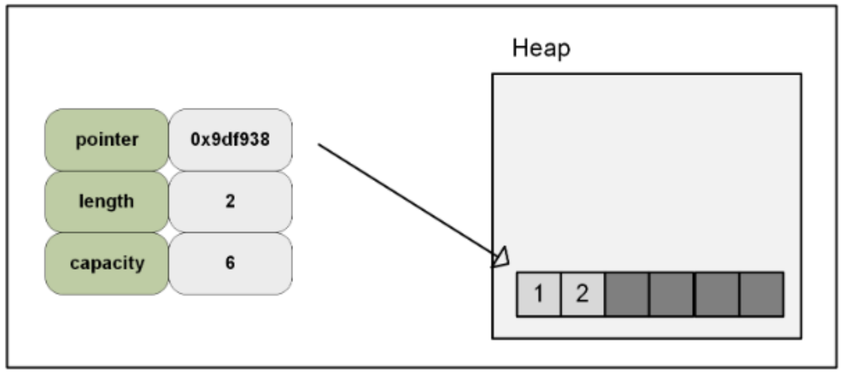
Then in addition to a pointer to the second element in `v`'s buffer, `s` also has an 8 byte length field with value 2:



The presence of the length field can also be seen in the following code in which the size of a slice(`&[i32]`) is 16 bytes (8 for the buffer pointer and 8 for the length field):

```rust
let mut vec: Vec<i32> = Vec::with_capacity(6);
vec.push(1);
vec.push(2);
```

We use `with_capacity`. Not required, though it is what Rust recommends whenever it is possible to specify how big the vector is expected to get. Directly after defining the vector, we have an empty vector with a lenght of 0 and a capacity of 6. Next, we push two elements onto it. Now, the vector contains the values `[ 1, 2]`. This increases the length to 2 while the capacity remains the same. Our vector now looks like this:



To better understand vectors, we can print several of it's properties to screen:

```rust
println!("----------------------------------------");
println!("Values inside vec: {:?}", vec);
println!("Length of the vec: {:?}", vec.len());
println!("Capacity of the vec: {:?}", vec.capacity());
println!("----------------------------------------");
```

The code outputs the following:

```
----------------------------------------
Values inside vec: [1, 2]
Length of the vec: 2
Capacity of the vec: 6
----------------------------------------
```

Here we display the values, the lenght and the capacity of the vector.

Now, if we were to add 5 elements to this vector, we would exceed the capacity and Rust would resize the vector. This resizing basically involves creating a new vector that has double the capacity and copying over the old vector. To see this in action, we can do the following:

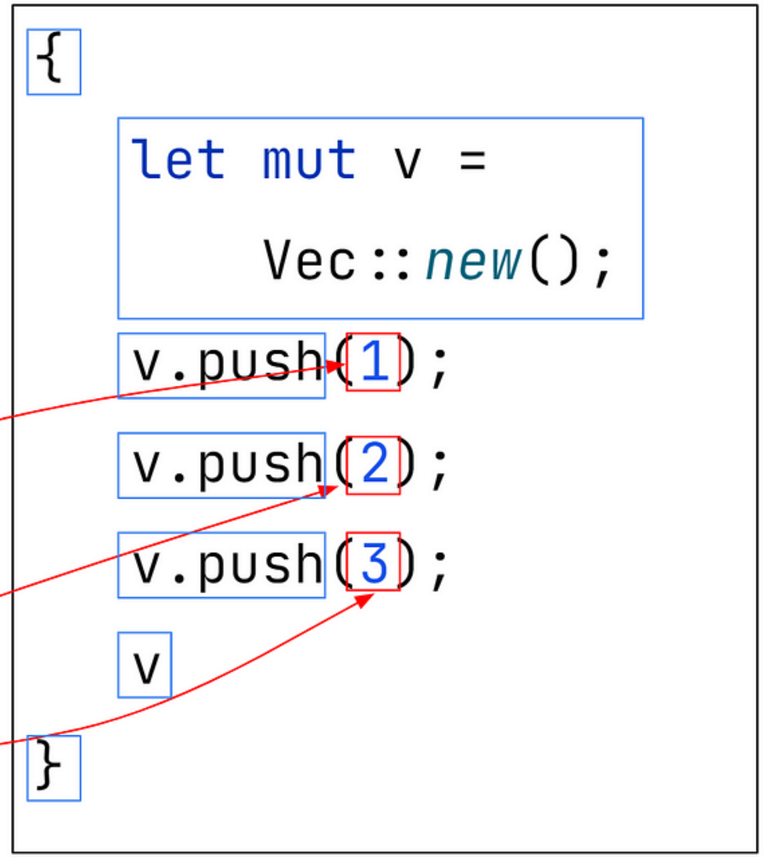
```rust
let mut vector = vec![3, 4, 5, 6, 7];
vec.append(&mut vector);
println!("----------------------------------------");
println!("Length of the vec: {:?}", vec.len());
println!("Capacity of the vec: {:?}", vec.capacity());
println!("----------------------------------------");
```

After adding this code, we would get to see the following:

```
----------------------------------------
Length of the vec: 7
Capacity of the vec: 12
----------------------------------------
```

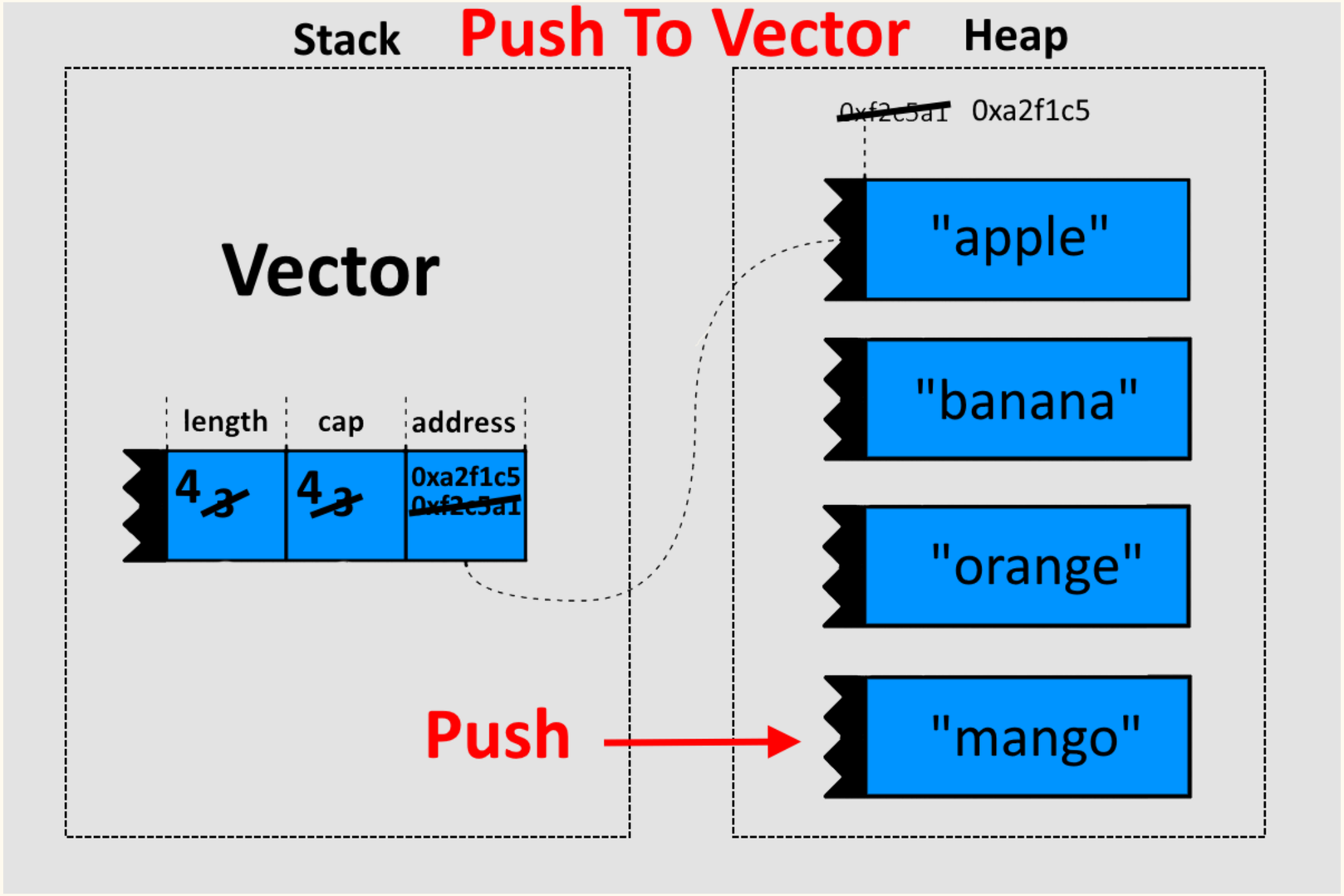
Common operations:

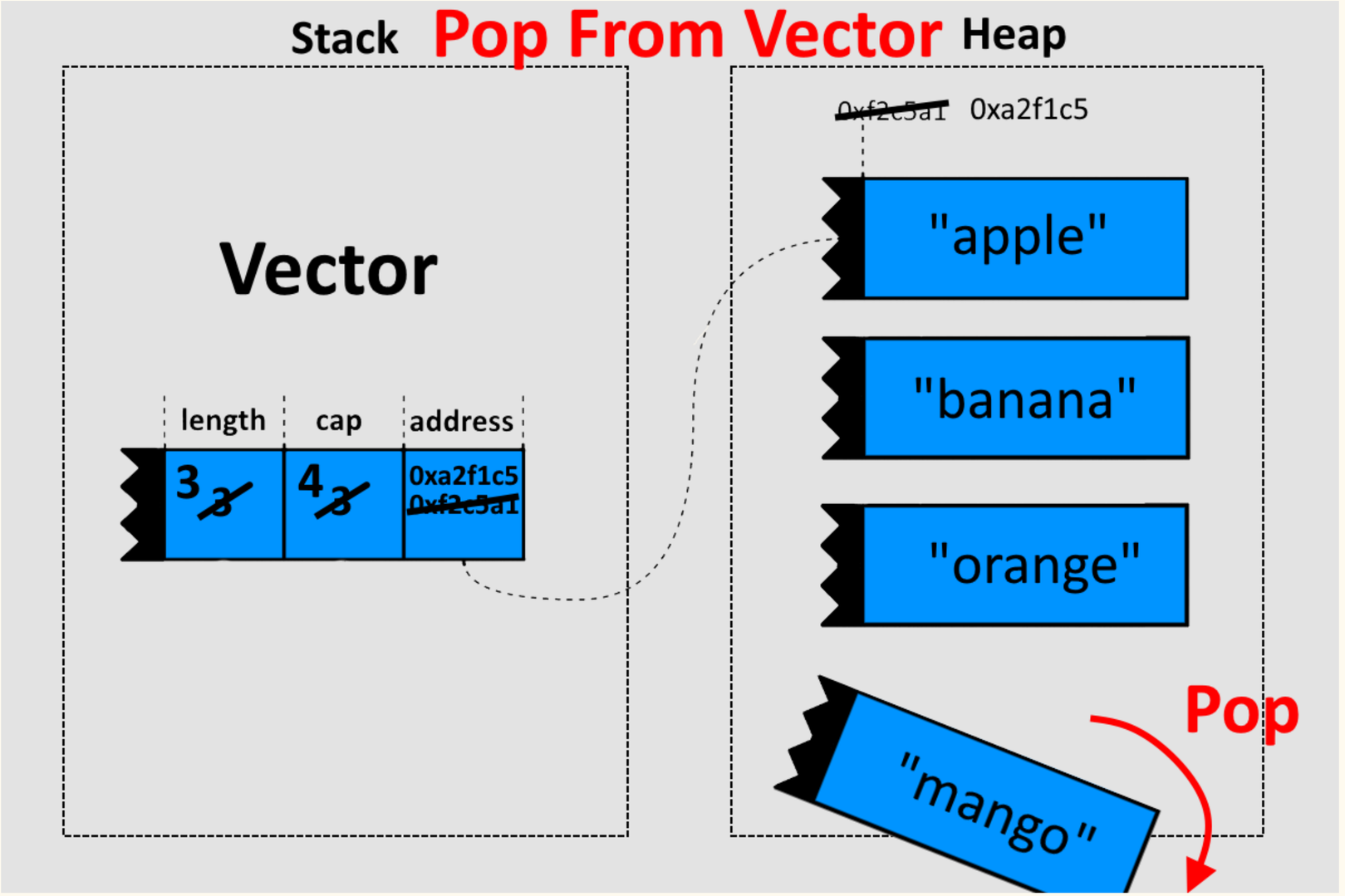
```
macro_rules! vec {
    ($($x:expr),+) => ({
        let mut v = Vec::new();
        $( v.push($x); )+
        v
    });
}

fn main() {
    let a = vec![1, 2, 3];
}
```



Vector mutations

```rust
let mut numbers: Vec<i32> = Vec::new();

numbers.push(1);
```


Stack
Push To Vector
Heap
Vector
length
cap
address
4
3
4
3
0xa2f1c5
0xf2c5a1
0xf2c5a1
0xa2f1c5
"apple"
"banana"
"orange"
"mango"
Push


Stack
Pop From Vector
Heap
Vector
length
cap
address
3
4
0xa2f1c5
0xf2c5a1
0xf2c5a1
0xa2f1c5
"apple"
"banana"
"orange"
"mango"
Pop

Exercice 3:
Vector mutations

II. HASHMAPS & HASHSETS

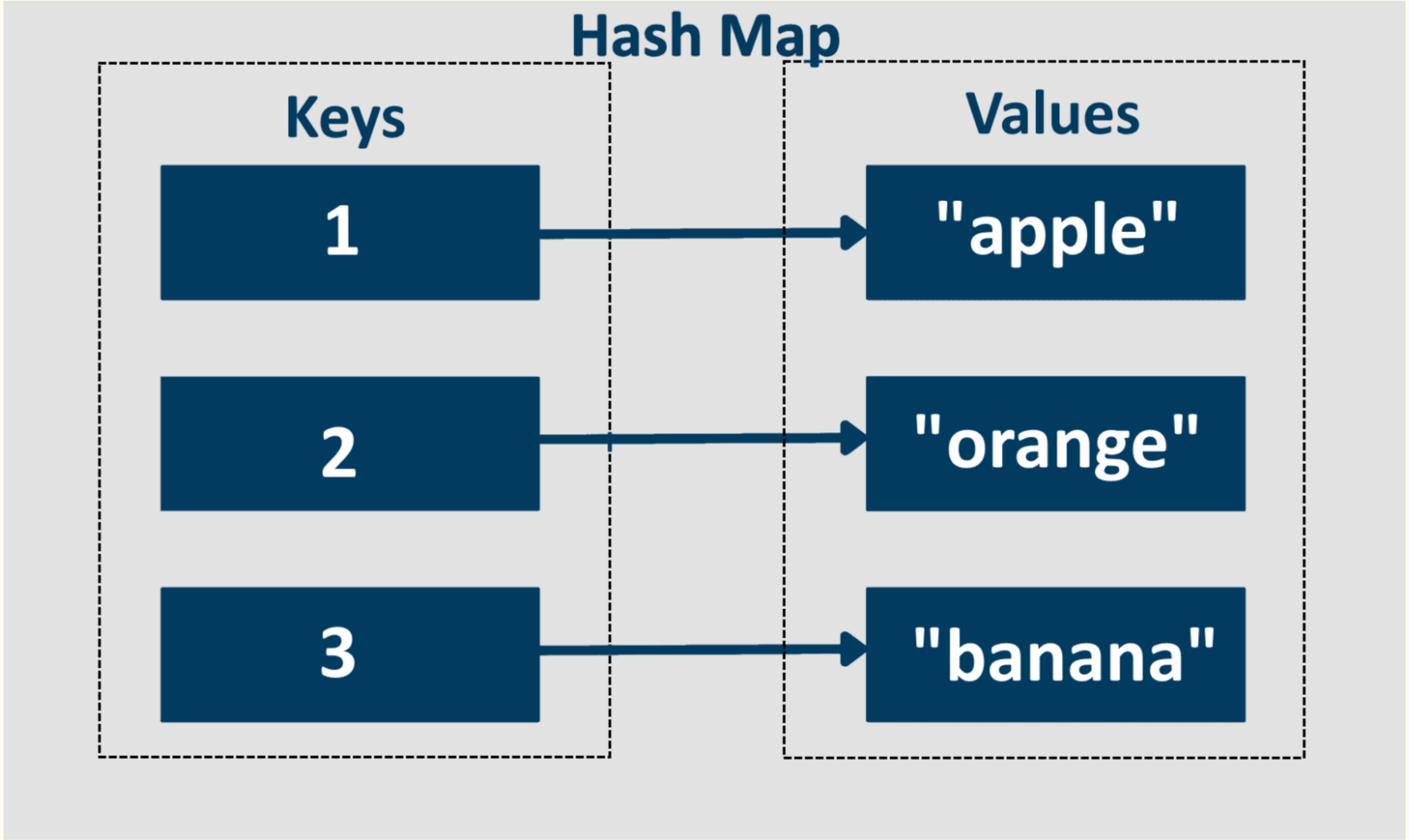
1. Hashmaps

Definition

A **HashMap** is an **unordered key-value store** in Rust that maps keys of type K to values of type V.

It consists of key **value-pairs**.


YOUR LOGO
MENU
RESTAURANT
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Mauris interdum est risus, convallis accumsan leo inter dum placerat.
STARTERS
TOMATO SOUP ..... $ 10.00
Lorem ipsum dolor sit amet, consectetur adipis cing elit mauris.
CHICKEN SOUP ..... $ 10.00
Lorem ipsum dolor sit amet, consectetur.
CRISPY CORN ..... $ 10.00
Lorem ipsum dolor sit amet, consectetur adipis cing elit mauris.
SALADS
GUACAMOLE SALAD ..... $ 10.00
Lorem ipsum dolor sit amet, consectetur adipis
CHICKEN SALAD ..... $ 10.00
Lorem ipsum dolor sit amet, conse ctetur adipiscing elit.
MAIN COURSES
GRILLED FISH AND POTATOES ..... $ 10.00
Lorem ipsum dolor sit amet, consectetur adipis cing elit mauris.
CHICKEN AND RICE ..... $ 10.00
Lorem ipsum dolor sit amet, consectetur adipis cing elit mauris.
TURKEY AND HAM PIE ..... $ 10.00
Lorem ipsum dolor sit amet, consectetur adipis cing elit mauris.
VEGETABLE PASTA ..... $ 10.00
Lorem ipsum dolor sit amet, consectetur adipis cing elit mauris.
DESSERTS
FRUIT AND CREAM ..... $ 5.00
Lorem ipsum dolor sit amet, consectetur.
ICE CREAM ..... $ 5.00
Lorem ipsum dolor sit amet, consectetur adi cing.
CHOCOLATE CAKE ..... $ 5.00
Lorem ipsum dolor sit amet.
STRAWBERRY CAKE ..... $ 5.00
Lorem ipsum dolor sit amet, consetur adipis cing.
APPLE PIE ..... $ 5.00
Lorem ipsum dolor sit amet.
WWW.RESTAURANTWEBSITE.COM
DRINKS
MINERAL WATER ..... $ 2.00
Lorem ipsum dolor sit amet.
FRESH FRUIT JUICE ..... $ 2.00
Lorem ipsum dolor sit amet.
COFFEE ..... $ 2.00
Lorem ipsum dolor sit amet.
TEA ..... $ 2.00
Lorem ipsum dolor sit amet.
WINES ..... $ 2.00
Lorem ipsum dolor sit amet.
DAILY FROM 12PM - 23PM
BOOK NOW
089 12 456 78


Hash Map
Keys
Values
1
"apple"
2
"orange"
3
"banana"

Exercice 4:
Hashmaps basics

Exercice 5:
Hashmaps access elements

2. Hashsets

Definition

A **Hashset** is a collection in Rust that stores **unique values** of **type T**, using a hash table for fast insertion, removal, and lookup.

It is backed by a **Hashmap<T, ()>** internally — the values are stored as keys, and the values are just empty ().

```rust
use std::collections::HashSet;

fn main() {
    let mut colors = HashSet::new();
    colors.insert("red");
    colors.insert("green");
    colors.insert("blue");
    colors.insert("blue"); // No duplicates

    println!("{:?}", colors);
    // {"green", "red", "blue"}
}
```

Exercice 6:
Hashsets basics

Your turn!

Sets operations